BASE DE DATOS

2026



Ing. Esteban Schab
AUS. Gabriel Casares
Aux. Alumno Tomas Viñuela

VISTAS

VISTAS

- Es un objeto que encapsula una sentencia SELECT.
- No almacena los datos físicamente por defecto; cada vez que se consulta la vista, el motor ejecuta la consulta subyacente.
- Se puede utilizar en las consultas de la misma manera que una tabla normal.

VISTAS - VENTAJAS

- **Simplificación**: Ocultan la complejidad de ciertas consultas.
- **Seguridad**: Permiten restringir el acceso a columnas o filas específicas, mostrando a cada usuario solo los datos permitidos.
- **Mantenimiento**: Centralizan la lógica de negocio para que no se repitan las mismas consultas en distintas aplicaciones.

VISTAS - SINTAXIS

```
CREATE VIEW nombre_vista AS
SELECT atributo1, atributo2
FROM tabla
WHERE condicion = 1;
```

FUNCIONES

FUNCIONES

(DEFINIDAS POR EL USUARIO)

- Son bloques de código reutilizables que permiten encapsular lógica personalizada para realizar cálculos, consultas o modificaciones de datos directamente en el servidor de la base de datos.

- Permiten extender las capacidades de SQL estándar. Permiten escribir lógica compleja que se ejecuta directamente donde están los datos, lo que reduce el tráfico de red entre las aplicaciones y la base de datos.

VENTAJAS

- Al residir en el servidor de BBDD ante cualquie modificación, no es necesario actualizar los programas que se conectan al mismo.
- Mayor eficiencia.
- Menor tráfico de red.
- Están pre-compilados.

LENGUAJES

- **SQL**:
  Ideal para consultas rápidas y simples.

- **PL/pgSQL**:
  Un lenguaje procedimental que permite usar variables, condiciones (IF) y bucles (LOOP).

- **Lenguajes externos**:
  Como Python, C o Perl (según la configuración de tu servidor)

FUNCIÓN BÁSICA SQL

```
create or replace function [nombre de
la función]([parámetros]) returns [tipo
de dato que retorna]
as
  [comandos sql]
language sql;
```

SINTAXIS GENERAL

```
CREATE [OR REPLACE] FUNCTION function_name (arguments)
RETURNS return_datatype AS $variable_name$
    DECLARE
        declaration;
        [...]
    BEGIN
        < function_body >
        [.. logic]
        RETURN { variable_name | value }
    END;
LANGUAGE language_name;
```

Si no se especifican los nombres de los parámetros, se los accede mediante $1, $2,..., $n

TIPOS DE DATOS DE RETORNO

- Tipos simples:
  Integer, char, text, varchar, real, numeric(10,2)
- Tipos compuestos definidos por TYPE
- Conjunto de valores:
  ```
  Returns setof tipodertorno
  ```
- Tablas definidas:
  ```
  Returns table (nombrecampo tipocampo...)
  ```

EJEMPLOS

```
1  CREATE FUNCTION addNumbers(val1 integer, val2 integer) RETURNS integer AS $$
2  BEGIN
3  RETURN val1 + val2;
4  END; $$
5  LANGUAGE PLPGSQL;
```

Data output | Messages | Notifications

```
CREATE FUNCTION

Query returned successfully in 174 msec.
```

```
7  select addnumbers(5, 6);
```

Data output | Messages | Notifications

	addnumbers integer
1	11

EJEMPLOS

```
1  create function get_stocks(price_cap int)
2  returns int
3  language plpgsql
4  as
5  $$
6  declare
7     stock_count integer;
8  begin
9     select count(*)
10    into stock_count
11    from public."Stocks"
12    where stock_price < price_cap;
13
14    return stock_count;
15 end;
16 $$;
```

Data output | Messages | Notifications

```
CREATE FUNCTION

Query returned successfully in 175 msec.
```

	stock_id [PK] integer	stock_name character varying	stock_price bigint
1	1	SBI Cards	950
2	2	XYZ Tyres	1016
3	3	Your Bank	99
4	4	Titan	450
5	5	TCS	3100

```
1  select get_stocks(1000);
```

Data output | Messages | Notifications

	get_stocks integer
1	3

EJEMPLOS

```
1  CREATE OR REPLACE FUNCTION get_allStocks (price_cap int)
2      RETURNS TABLE (
3          stock_serial_no int,
4          stock varchar,
5          price_of_stock bigint
6  )
7  AS $$
8  BEGIN
9      RETURN QUERY SELECT
10         stock_id,
11         stock_name,
12         stock_price
13     FROM
14         public."Stocks"
15     WHERE
16         stock_price < price_cap;
17 END; $$
18
19 LANGUAGE 'plpgsql';
```

Data output | Messages | Notifications

```
CREATE FUNCTION

Query returned successfully in 482 msec.
```

```
1  select get_allstocks(1000);
```

Data output | Messages | Notifications

	get_allstocks record
1	(1,"SBI Cards",950)
2	(3,"Your Bank",99)
3	(4,Titan,450)

DOCUMENTACIÓN PL/PGSQL

https://www.postgresql.org/docs/current/plpgsql.html

TRIGGERS

TRIGGERS

- Un TRIGGER, es un 'disparador' que se ejecuta cuando queremos hacer una operación en una tabla, por ejemplo (inserción, borrado, modificación).

- Es un bloque de código, inserto en una función que es utilizado para comprobar la integridad de los datos y que la base de datos contenga al final de una transaccion un estado correcto, conforme lo previamente establecido.

TRIGGERS - VENTAJAS

- Mantener la integridad de los datos.
- Garantizar que se ejecute para cada operación que se realiza en una tabla.
- Facilidad de implementar.
- Facilidad para modificar cuando cambian las reglas de negocio.
- Mantener la independencia de los datos.

TUPLAS OLD Y NEW

- Los triggers nos permiten contar con dos TUPLAS VIRTUALES
- NEW contiene los valores de los atributos para la tupla que se quiere insertar o modificar.
- OLD contiene los valores que están actualmente almacenados en la tabla.
- NEW.atributo u OLD.atributo obtenemos el valor del atributo para esa tupla.

EJECUCIÓN - AFTER O BEFORE

- Al definir un disparador se puede indicar que se ejecute antes (before) de producirse alguna opreación sobre la tabla (INSERTE, DELETE o UPDATE) ó bien después (after) de realizada la operación.

- ¿cuál sería la diferencia? ¿Cuándo utilizar AFTER y cuándo BEFORE?

SINTAXIS

```
CREATE FUNCTION nombre_funcion() RETURNS TRIGGER
AS $$
BEGIN

Sentencias que se vayan a ejecutar.

END;
$$ LANGUAGE plpgsql;
```

SINTAXIS

```
CREATE TRIGGER nombre_trigger

BEFORE INSERT ON tabla

FOR EACH ROW EXECUTE PROCEDURE nombre _funcion;
```

EJEMPLO

Tenemos una tabla emplea con los siguientes datos:

```
CREATE TABLE emplea (
dni int NOT NULL,
apellido character varying(32),
nombre CHARACTER VARYING(25),
fecNac date,
Constraint emplea_pkey Primary Key (dni)
);
```

EJEMPLO - RESTRICCIONES

- Que el atributo apellido no sea nulo
- Que el empleado sea mayor de 18 años.

EJEMPLO - FUNCIÓN

```
CREATE OR REPLACE FUNCTION func_e() RETURNS TRIGGER AS
$funcemp$
DECLARE
edad smallint ;
BEGIN
edad := date_part('year',age(NEW.fecNac));
IF NEW.apellido IS NULL THEN
        RAISE EXCEPTION 'no puede tener apellido vacío';
END IF;

IF edad <= '18' THEN
        RAISE EXCEPTION 'no puede ser menor de 18 años';
END IF;
  RETURN NEW;
END;
$funcemp$
LANGUAGE plpgsql;
```

EJEMPLO - TRIGGER

```
CREATE TRIGGER trigger_e
BEFORE INSERT OR UPDATE ON emplea
FOR EACH ROW EXECUTE PROCEDURE
func_e();
```

EJEMPLO – PRUEBAS

- INSERT INTO emplea VALUES('11111111', 'GONZALEZ', 'JUAN','2008-11-10');
  ERROR: no puede ser menor de 18 años

- INSERT INTO emplea VALUES('11111111',null, 'JUAN', '1950-11-10');
  ERROR: no puede tener apellido vacío

- INSERT INTO emplea VALUES('111111111', 'GONZALEZ', 'JUAN', 1978-11-10');
  Query returned successfully: one row affected, 1028 ms execution time.